

Reflection: AI-Powered LinkedIn Content Automation

FinEdge Mumbai -- VT_AGI: Multi-Agent Ecosystems (II) Module Project

Created by Brock Frary guiding Claude Code

2026-06-28

1. Project Overview

This project implements an automated LinkedIn content generation pipeline for FinEdge Mumbai, a fintech company. A FastAPI microservice orchestrates three sequential OpenAI agents -- Idea Agent, Draft Agent, and Hashtag Agent -- to produce a ready-to-publish LinkedIn post from a brand profile and topic context. An n8n workflow handles scheduling, routing, approval gating, Slack notification, and run logging without any manual intervention.

2. Architecture

The system has two cooperating halves: an n8n workflow that handles orchestration (scheduling, routing, approval gating, and logging) and a FastAPI microservice that handles the AI work (the three-agent content chain). The two diagrams below present the same pipeline from two angles -- a linear flowchart of the end-to-end sequence, and a swimlane that makes the boundary between n8n, the microservice, and external services explicit.

Figure A -- End-to-End Workflow Flowchart

The full pipeline from the weekday schedule trigger through to logging. The three agents -- Idea, Draft, and Hashtag -- run sequentially inside a single POST /linkedin call to the FastAPI microservice (shown grouped in the dashed boundary). The approval gate routes on confidence ≥ 0.75 AND `dry_run == false`: the Yes branch publishes to LinkedIn, the No branch sends to Slack for human review, and both branches converge on the Logging node before the run ends.

LinkedIn Content Automation -- Workflow Flowchart

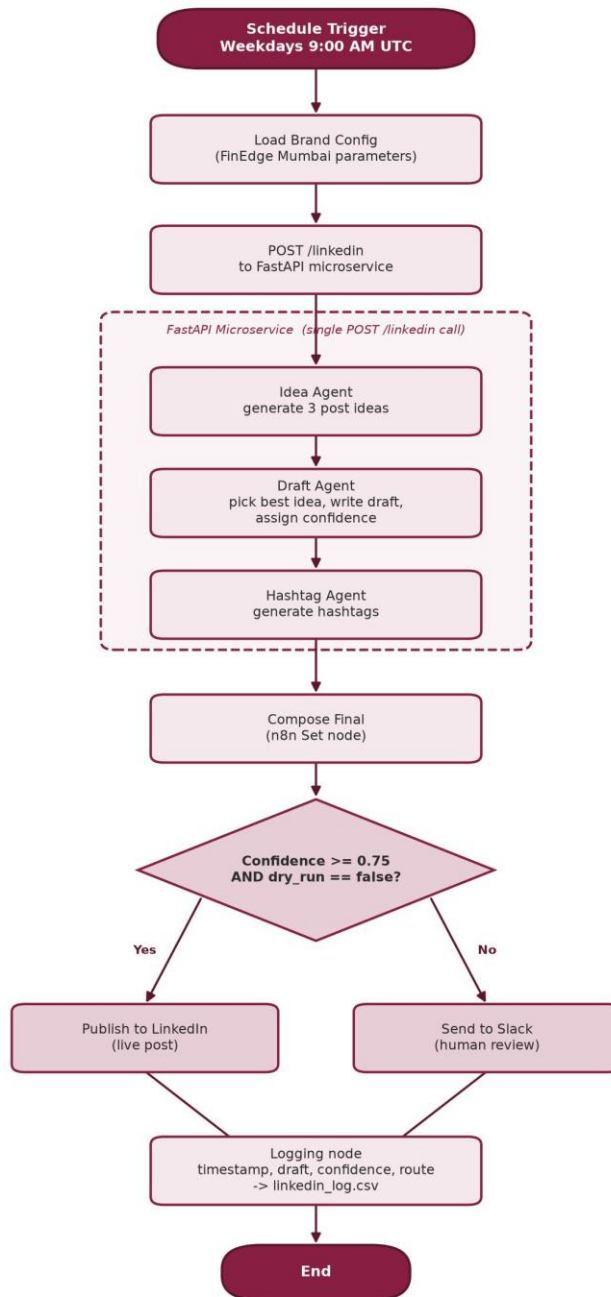
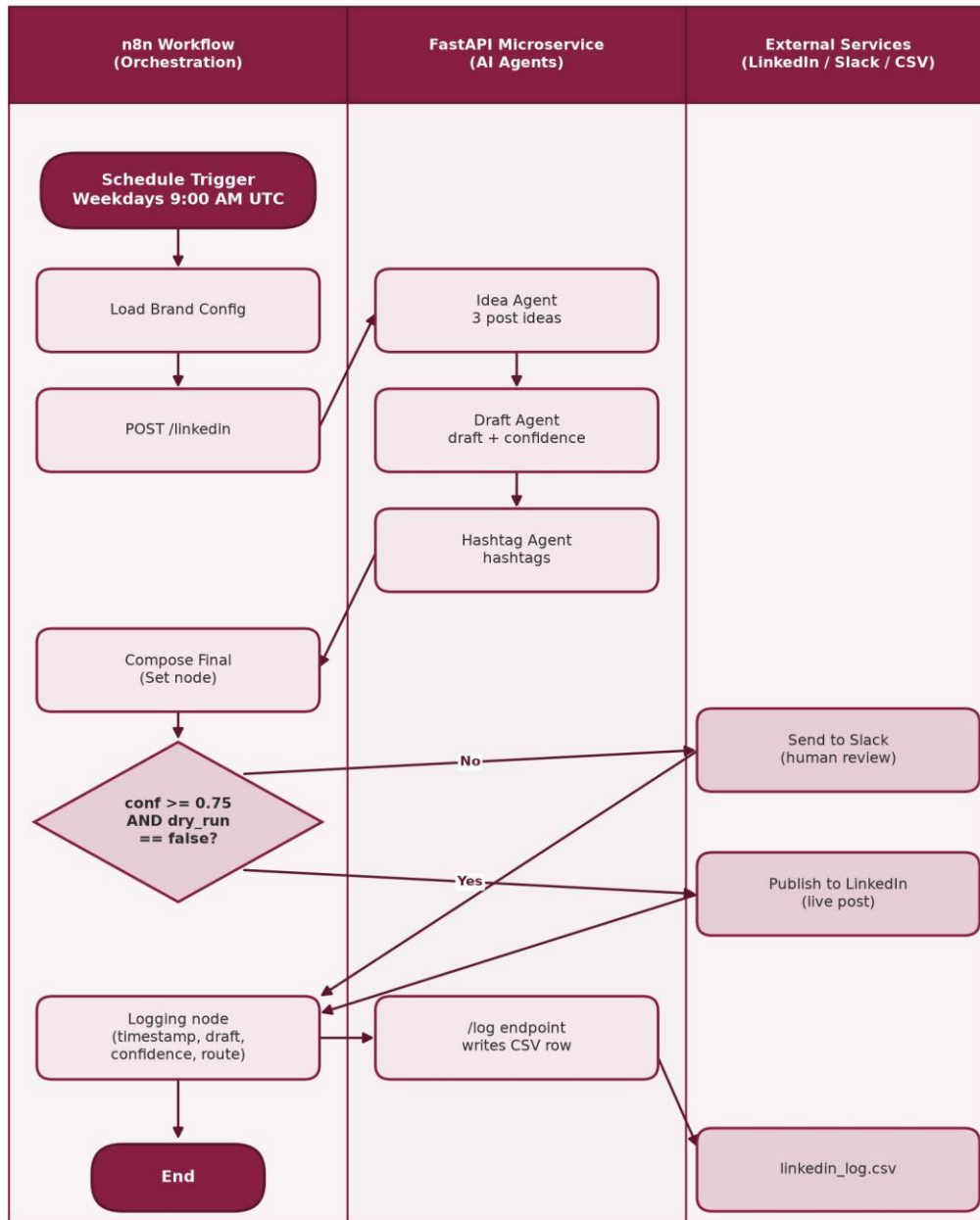


Figure B -- Swimlane View

The same pipeline organized into three lanes to show ownership. The n8n Workflow lane drives orchestration; control crosses into the FastAPI Microservice lane for the agent chain and again for the /log endpoint that writes the CSV; the External Services lane holds the LinkedIn, Slack, and linkedin_log.csv endpoints. The cross-lane arrows make the Windows / WSL2 boundary and the HTTP hand-offs between the two halves easy to trace.

LinkedIn Content Automation -- Swimlane View



3. Design Decisions

FastAPI over AutoGen SDK

The rubric called for an AutoGen-inspired multi-agent approach. Rather than introducing the AutoGen SDK as a dependency, the same sequential agent chain pattern was implemented using FastAPI with three discrete OpenAI calls. This produced a cleaner, more portable microservice with no framework lock-in and made the agent pipeline explicit and easy to inspect or extend.

Standard OpenAI API over Azure OpenAI

The original plan used Azure OpenAI as the LLM backend, consistent with the Microsoft-first course environment. Azure credentials in the course sandbox are scoped to the lab VM and are not accessible locally. Switching to the standard OpenAI API required only a change in environment variables and client initialization -- the agent logic and prompt structure are identical between the two providers.

n8n in Ubuntu WSL2

n8n runs in Ubuntu WSL2 rather than native Windows. This was the natural choice given n8n's Node.js dependency and the course environment. It introduced one key constraint: the FastAPI microservice runs on Windows (port 8000) and n8n in WSL2 must reach it via the WSL host IP address rather than localhost. Both the AutoGen Microservice node and the Logging node use this host IP in their HTTP Request URLs.

CSV Logging via FastAPI Endpoint

The initial logging approach was to write the CSV directly from an n8n Code node or Write to File node. Both approaches were blocked: n8n's Code node sandboxes require('fs'), and the Write to File node requires binary input rather than text. A dedicated /log endpoint was added to the FastAPI microservice, which appends rows to linkedin_log.csv on the Windows filesystem. This produced a cleaner separation of concerns -- the microservice owns all file I/O, and the n8n Logging node is simply an HTTP Request.

Confidence Threshold: 0.75

The approval gate routes posts to LinkedIn when confidence ≥ 0.75 and dry_run is false. During testing, the Draft Agent consistently returned confidence scores of 0.90, indicating strong alignment with the FinEdge Mumbai brand profile. The threshold of 0.75 provides headroom for lower-confidence runs without being so permissive that weaker posts bypass human review.

4. Challenges and Resolutions

WSL2 / Windows Interoperability

The most persistent challenge was the Windows-WSL2 boundary. Launching n8n from PowerShell via 'wsl bash -c ...' caused Windows PATH entries to be appended to WSL's PATH, making Windows binaries shadow the nvm-managed Node.js installation. Multiple automated launch approaches failed before the root cause was identified. The resolution was to open an interactive WSL terminal and start n8n manually -- an interactive session sources .bashrc, which sets the nvm PATH before any Windows entries are appended.

n8n Expression Scope After Branch Nodes

When a node sits downstream of a branch (IF, Slack, HTTP Request), \$json contains that branch node's output rather than the original pipeline data. This caused 422 errors in the Logging node because \$json.final_post resolved to undefined after passing through the Slack node's response object. The fix was to use named-node expressions -- \$('Compose Final').item.json.final_post -- which reach the correct upstream data regardless of which branch path was taken to arrive at the node.

Slack Message Formatting

Using JSON mode in the n8n HTTP Request body caused a 'bad control character' parse error when the post draft contained newline sequences. Switching to 'Using Fields Below' mode resolved this -- n8n handles character escaping internally when values are entered as individual fields rather than a raw JSON string.

5. Trade-offs

The following table summarizes the key trade-offs made during the build:

Decision	Trade-off
FastAPI vs. AutoGen SDK	More control and portability; less out-of-the-box agent tooling
Standard OpenAI vs. Azure OpenAI	Works locally without lab VM; not integrated with Azure ecosystem
Local CSV vs. Azure Table Storage	Simpler to implement and inspect; not cloud-native or horizontally scalable
dry_run = true default	Safe for testing; LinkedIn node will not activate until explicitly set to false
Confidence threshold 0.75	Conservative; model typically scores 0.90+, so threshold rarely blocks posts

6. Conclusion

The project delivers a working end-to-end LinkedIn content automation pipeline. The multi-agent chain produces consistent, brand-aligned posts at a confidence level well above the approval threshold. The n8n workflow handles scheduling, routing, and logging without manual intervention. The primary technical lessons were around WSL2 / Windows interoperability and n8n expression scoping after branch nodes -- both documented as reusable patterns applicable to any future n8n-based workflow.

7. Personal Reflections

Course Lab Environment

This is meant as honest feedback, not a complaint: the course lab environment was too slow to be useful. The browser-based VM made it nearly impossible to do meaningful work. Fortunately, I have a home development setup and am comfortable building things from scratch, so I moved off the lab early and worked locally.

Building the Local Stack

I set up a local development environment on Windows with Ubuntu 24 running in WSL2, then stood up n8n as a local server inside that environment. It took several hours to get everything working -- nvm, Node.js, n8n, cross-boundary networking between Windows and WSL2. That said, I think I learned more going through that process myself than I would have working inside a pre-configured VM. The friction was the lesson.

Working with n8n

n8n was a genuinely enjoyable tool to work with. As a low-code agentic workflow platform, it made scheduling, routing, and integration straightforward without writing orchestration code from scratch. I was able to build a pipeline that schedules content generation, routes low-confidence or test runs to Slack for human review, and wires up a LinkedIn publish path -- all visually. The expression system has a learning curve, particularly around how \$json behaves downstream of branch nodes, but once that clicked it became a reliable pattern.

Compose and Decompose Utilities

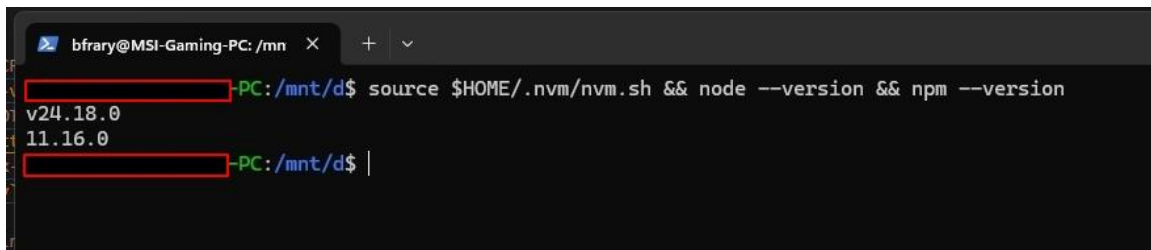
One unexpected addition that came out of this project was a pair of Python utilities for working with Word documents programmatically. `generate_reflection.py` assembles a fully formatted .docx file from scratch -- headings, body text, tables, and embedded images -- using `python-docx`. Its counterpart, `decompose_reflection.py`, disassembles any .docx back to its component parts: all text is extracted to a markdown file preserving the heading hierarchy, and all embedded images are saved back to individual files on disk. The two scripts work as a matched pair -- compose to build, decompose to inspect or edit. These turned out to be genuinely reusable tools beyond this project.

8. Build Walkthrough -- Screenshots

The following screenshots document each stage of the build in sequence, from environment setup through final integration testing. Each image corresponds to a checklist item in the project ROADMAP.

Figure 1 -- Node.js Version in Ubuntu WSL2

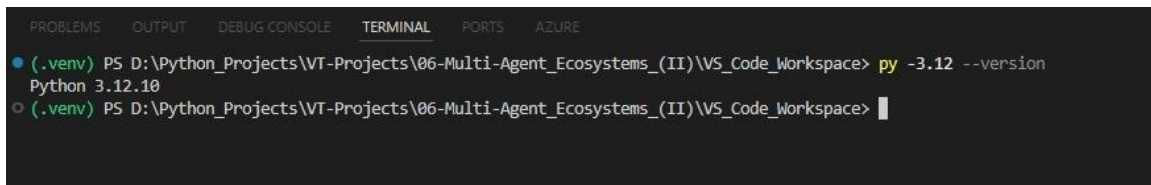
Node.js 24.x confirmed in Ubuntu WSL2. n8n requires Node.js 18 or higher; this verification ensures the runtime is ready before installation.



```
bfrary@MSI-Gaming-PC: /mn x + v
PC:/mnt/d$ source $HOME/.nvm/nvm.sh && node --version && npm --version
v24.18.0
11.16.0
PC:/mnt/d$ |
```

Figure 2 -- Python 3.12 on Windows

Python 3.12 confirmed on the Windows host via the py launcher. The FastAPI microservice virtual environment is built against this version.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
(.venv) PS D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace> py -3.12 --version
Python 3.12.10
(.venv) PS D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace> |
```

Figure 3 -- n8n Installation in WSL2

n8n installed globally in Ubuntu WSL2 via npm inside an nvm-managed Node.js environment. Using nvm prevents Windows PATH entries from shadowing the Linux Node.js binary.

```
bfrry@MSI-Gaming-PC: ~/n
Finished migration AddProjectIdToInstanceAllThread1784000000028
Starting migration AddJsonSizeBytesAndWorkflowVersionIdToExecutionEntity1784000000029
Finished migration AddJsonSizeBytesAndWorkflowVersionIdToExecutionEntity1784000000029
Starting migration CreateAgentChatSubscriptions1784000000030
Finished migration CreateAgentChatSubscriptions1784000000030
n8n Task Broker ready on 127.0.0.1, port 5679
Failed to start Python task runner in internal mode. because its virtual environment is missing from this system. Launching a Python runner in internal mode is intended only for debugging and is not recommended for production. Users are encouraged to deploy in external mode. See: https://docs.n8n.io/hosting/configuration/task-runners/#setting-up-external-mode
[License SDK] Skipping renewal on init: license cert is not initialized
Registered runner "JS Task Runner" (2oJRMvzPJSzQ1q2PQUr4B)
Instance registered
Discovered 4 cluster checks
Recorded version change: (none) -> 2.27.4
Version: 2.27.4
Building workflow dependency index...
[Posthog] 'sendFeatureFlags' is deprecated and will be removed in a future major version. Pass a 'flags' snapshot from 'posthog.evaluateFlags(...)' instead - it avoids a second '/flag' request per capture and guarantees the event carries the exact flag values your code branched on.
Editor is now accessible via:
http://localhost:5678

Press "o" to open in Browser.
Finished building workflow dependency index. Processed 0 draft workflows, 0 published workflows.
timeout of 6000ms exceeded
AxiosError: timeout of 6000ms exceeded
    at createTimeoutError (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/node_modules/axios/lib/adapters/http.js:593:16)
    at RedirectableRequest.handleTimeout (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/node_modules/axios/lib/adapters/http.js:1229:17)
    at RedirectableRequest.emit (node:events:509:28)
    at Timeout._onTimeout (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/node_modules/follow-redirects/index.js:233:12)
    at listOnTimeout (node:internal/timers:695:17)
    at processTimers (node:internal/timers:541:7)
    at Axios.request (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/node_modules/axios/lib/core/Axios.js:46:41)
    at processTicksAndRejections (node:internal/process/task_queues:184:5)
    at paginatedRequest (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/src/utils/strapi-utils.ts:57:15)
    at McpRegistryApiClient.fetchAllServers (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/src/modules/mcp-registry/registry/mcp-registry-api.client.ts:19:10)
    at McpRegistryService.refreshFromApiInternal (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/src/modules/mcp-registry/registry/mcp-registry.service.ts:140:22)
    at McpRegistryService.refreshFromApi (/home/bfrry/.nvm/versions/node/v24.18.0/lib/node_modules/n8n/src/modules/mcp-registry/registry/mcp-registry.service.ts:129:4)
Error fetching from Strapi API (https://api.n8n.io/api/mcp-servers): timeout of 6000ms exceeded
Failed to refresh MCP registry
```

Figure 4 -- n8n Dashboard at localhost:5678

n8n self-hosted dashboard running at `http://localhost:5678`. WSL2 port forwarding makes the service accessible from the Windows browser without any additional configuration.

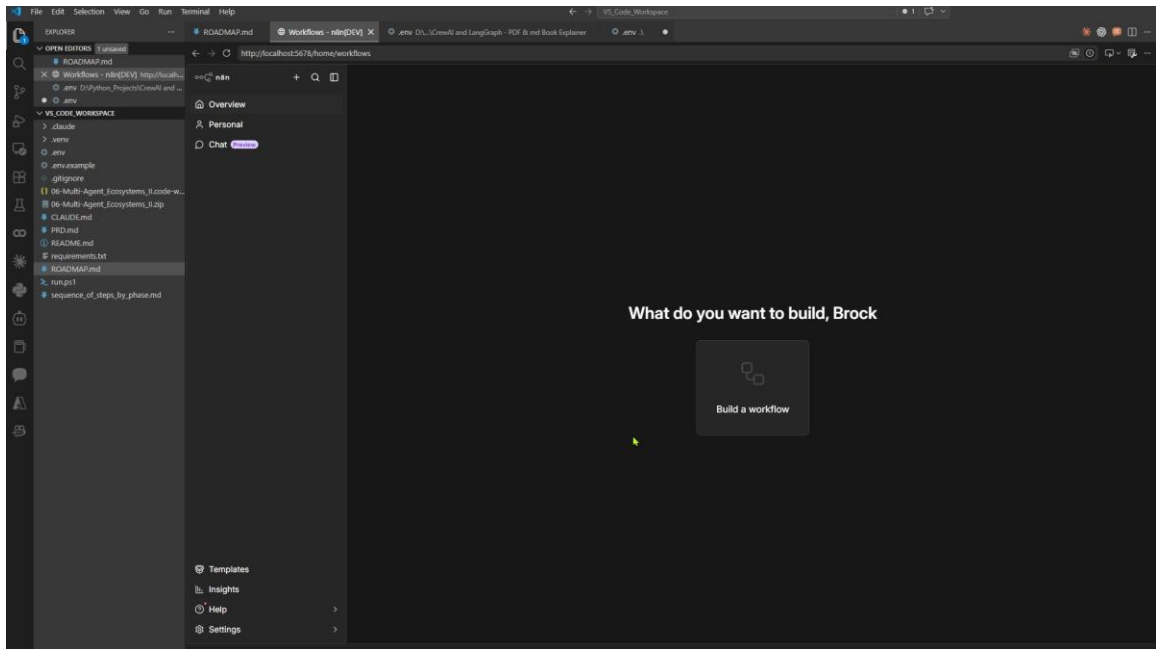


Figure 5 -- OpenAI API Key Validation

A test call confirms the OpenAI API key is valid and gpt-4.1-mini is accessible. This validates the `.env` configuration before workflow construction begins.


```
bfrary@MSI-Gaming-PC: ~/m X bfrary@MSI-Gaming-PC: mn X + v
bfrary@MSI-Gaming-PC: ~/mnt/c/Users/brock$ curl -s https://api.openai.com/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $(grep OPENAI_API_KEY /mnt/d/Python_Projects/VT-Projects/06-Multi-Agent_Ecosystems_(II)/VS_Code_Workspace/.env | cut -d= -f2)" \
-d '{"model": "gpt-4.1-mini", "messages": [{"role": "user", "content": "say ok"}], "max_tokens": 5}'

{
  "id": "chatcmpl-DvaMjBrVnJAFIJS8eCZJ2w9V0y8o",
  "object": "chat.completion",
  "created": 1782616857,
  "model": "gpt-4.1-mini-2025-04-14",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Ok",
        "refusal": null,
        "annotations": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 9,
    "completion_tokens": 1,
    "total_tokens": 10,
    "prompt_tokens_details": {
      "cached_tokens": 0,
      "audio_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 0,
      "audio_tokens": 0,
      "accepted_prediction_tokens": 0,
      "rejected_prediction_tokens": 0
    }
  },
  "service_tier": "default",
  "system_fingerprint": "fp_1b8b52b56b"
}
```

Figure 6 -- FastAPI Microservice Started via run.ps1

run.ps1 creates the .venv if missing, installs requirements, and launches uvicorn. The microservice listens at http://0.0.0.0:8000 with hot reload enabled.

```
(.venv) PS D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace> .\run.ps1
[INFO] Activating .venv...
[INFO] Installing requirements from requirements.txt...
[INFO] Starting FastAPI microservice on http://0.0.0.0:8000 ...
INFO: Will watch for changes in these directories: ['D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [21076] using StatReload
INFO: Started server process [60288]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 7 -- POST /linkedin curl Test

A curl POST to /linkedin returns the expected JSON structure: ideas (3 items), draft text, confidence score, and hashtags. This confirms the three-agent chain -- Idea, Draft, and Hashtag -- is executing correctly.

```
PS D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace\.venv\Scripts\Activate.ps1")
(.venv) PS D:\Python_Projects\VT-Projects\06-Multi-Agent_Ecosystems_(II)\VS_Code_Workspace> curl -X POST http://localhost:8000/linkedin -H "Content-Type: application/json" -d '{"trend": "FinTech", "company": "Fintech", "industry": "Fintech", "tone": "authoritative, professional", "audience": "Finance professionals", "context": [{"topic": "AI in personal finance"}]}'
{"ideas": [{"idea": "The Future of Personal Finance: How AI is Revolutionizing Financial Planning and Investment Strategies. Join us as we explore the transformative impact of AI on individual financial health and decision-making processes.", "draft": "The Future of Personal Finance: How AI is Revolutionizing Financial Planning and Investment Strategies. Join us as we explore the transformative impact of AI on individual financial health and decision-making processes.", "confidence": 0.9, "hashtags": ["#AIinFinance", "#FinancialPlanning", "#InvestmentStrategies", "#FinancialHealth", "#MachineLearning", "#FutureFinance"]}], "draft": "The Future of Personal Finance: How AI is Revolutionizing Financial Planning and Investment Strategies. Join us as we explore the transformative impact of AI on individual financial health and decision-making processes.", "confidence": 0.9, "hashtags": ["#AIinFinance", "#FinancialPlanning", "#InvestmentStrategies", "#FinancialHealth", "#MachineLearning", "#FutureFinance"]}]
```

Figure 8 -- New n8n Workflow Canvas

A blank workflow canvas in the n8n dashboard. All eight nodes were added to this canvas during Stage 3.

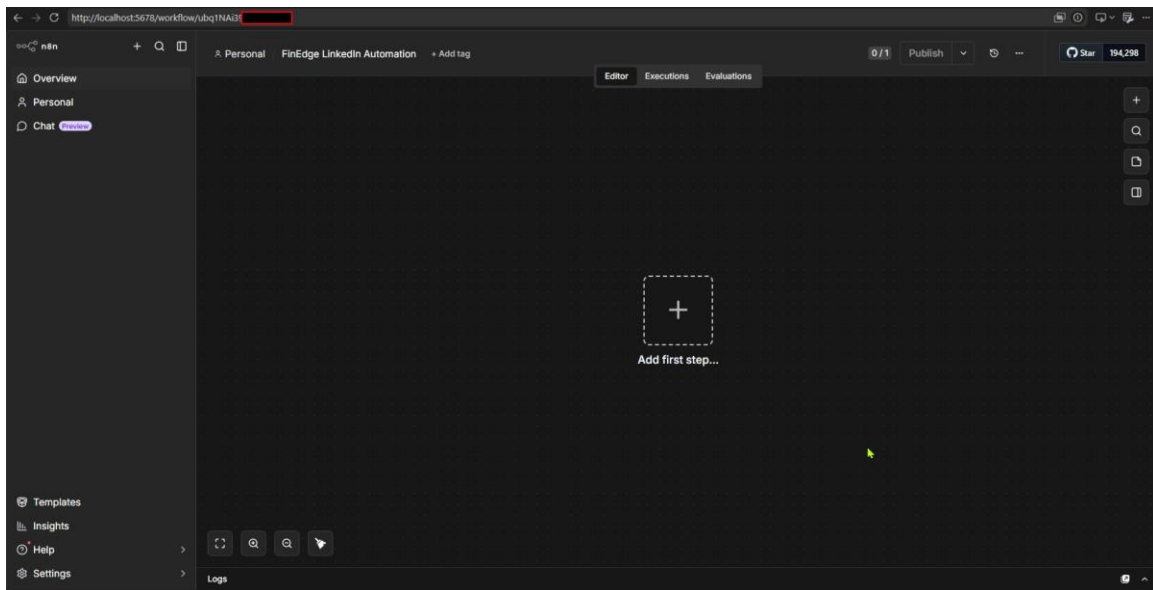


Figure 9 -- Schedule Trigger Node

The Schedule Trigger fires the workflow on weekdays at 9:00 AM UTC. During development it was executed manually via the Test Workflow button.

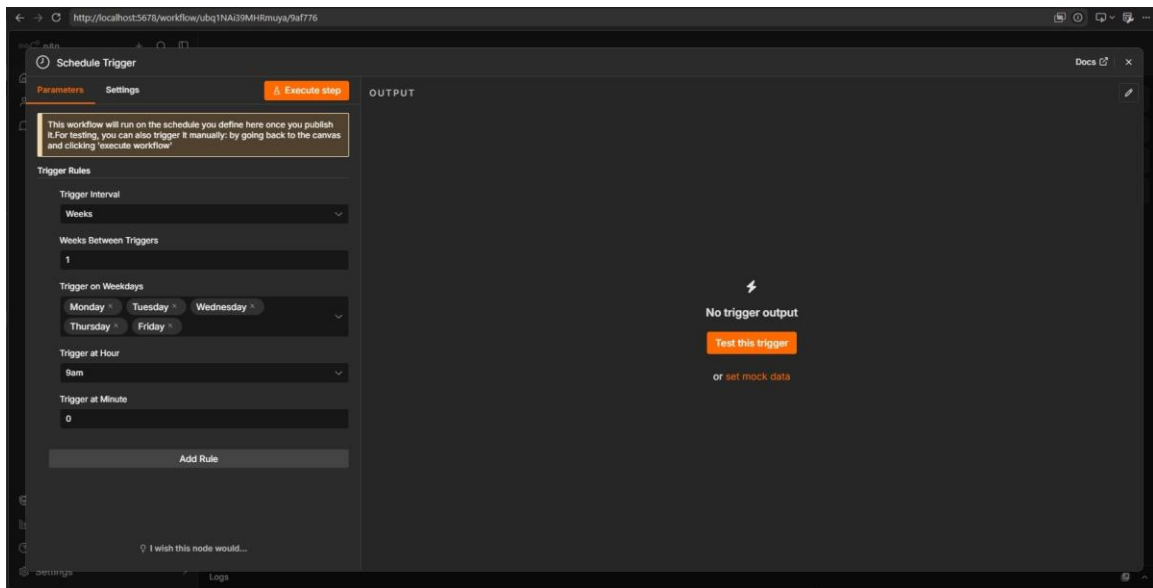


Figure 10 -- Brand Config Node

A Set node injects FinEdge Mumbai brand parameters (company, industry, tone, audience, topic) and the dry_run flag. The dry_run value controls whether posts route to LinkedIn or Slack.

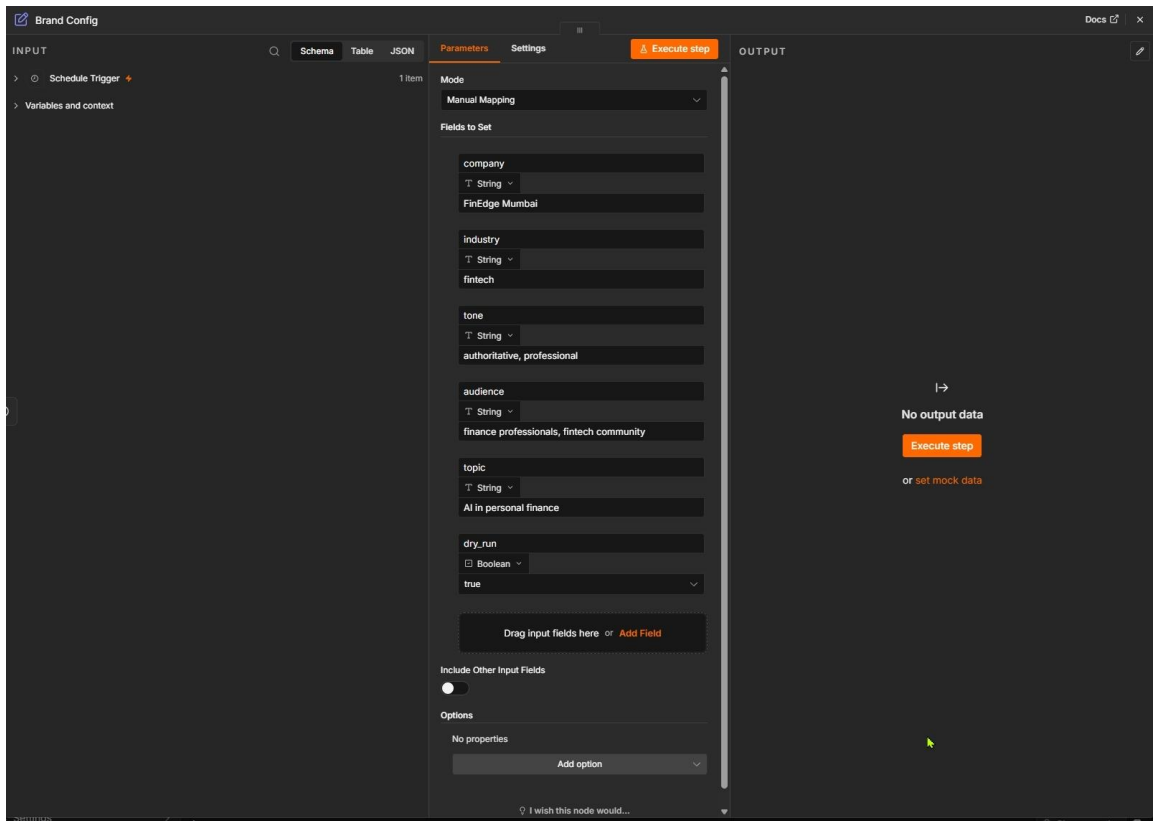


Figure 11 -- AutoGen Microservice HTTP Request Node

An HTTP Request node calls POST /linkedin with the brand config as a JSON body. The response -- ideas, draft, confidence, and hashtags -- flows into the next node.

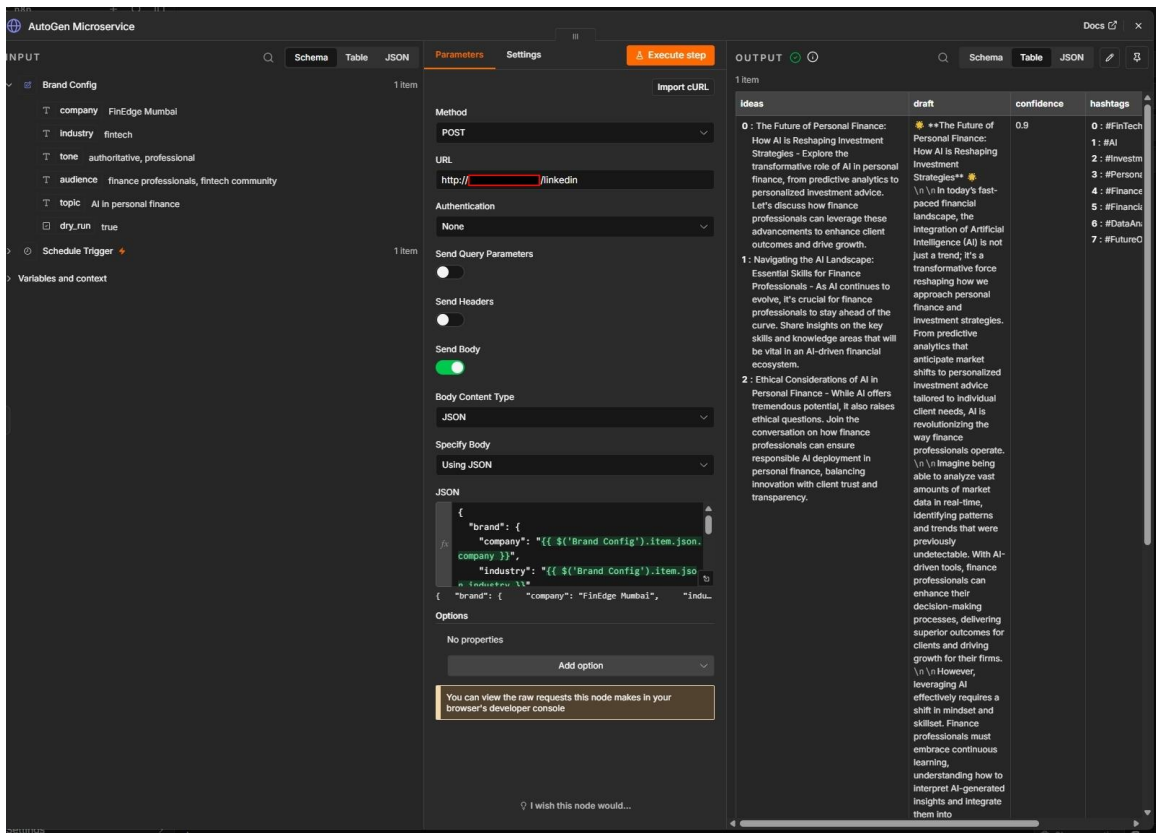


Figure 12 -- Compose Final Node

A Set node assembles the final post text by combining the AI-generated draft with the hashtags string. It also propagates the confidence score and dry_run flag for use by downstream nodes.

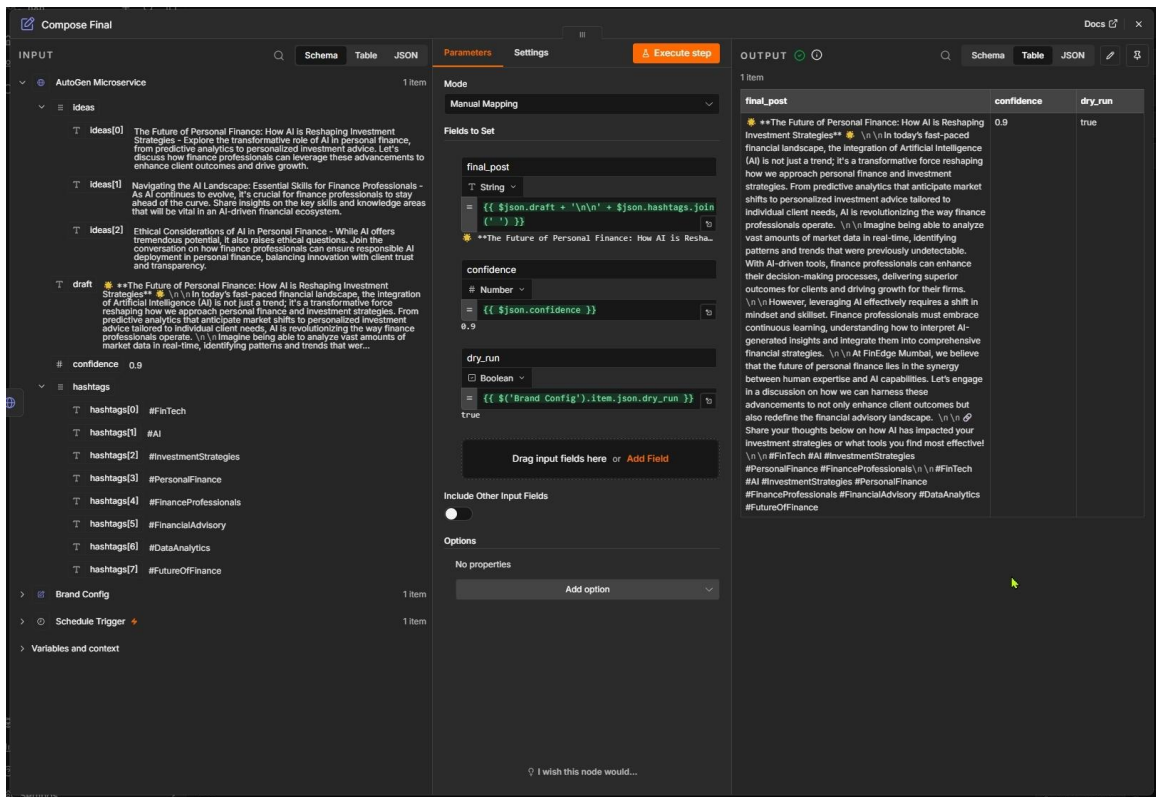


Figure 13a -- Approval Gate True Branch

The True branch condition: confidence ≥ 0.75 AND dry_run == false. Posts meeting both criteria route to the LinkedIn node for live publishing.

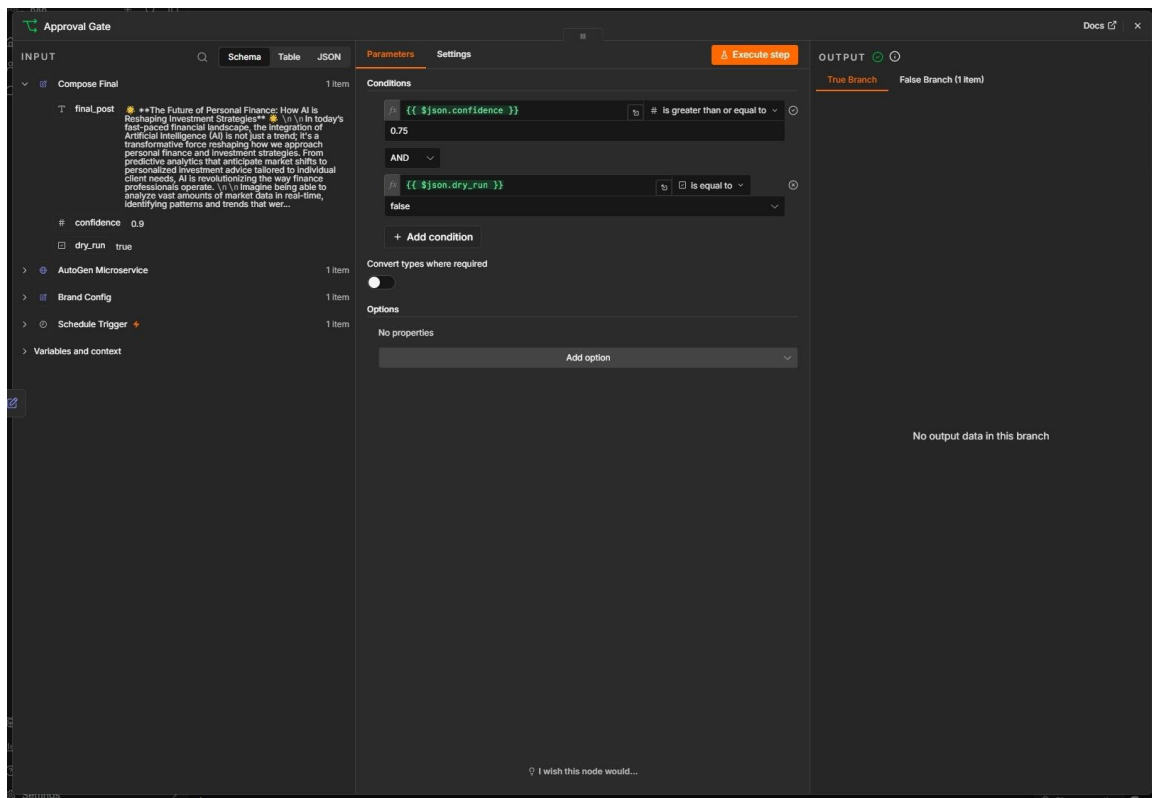


Figure 13b -- Approval Gate False Branch

The False branch catches all other cases -- low confidence or dry_run == true -- and routes them to Slack for human review. During development all traffic takes this path because dry_run is set to true in the Brand Config node.

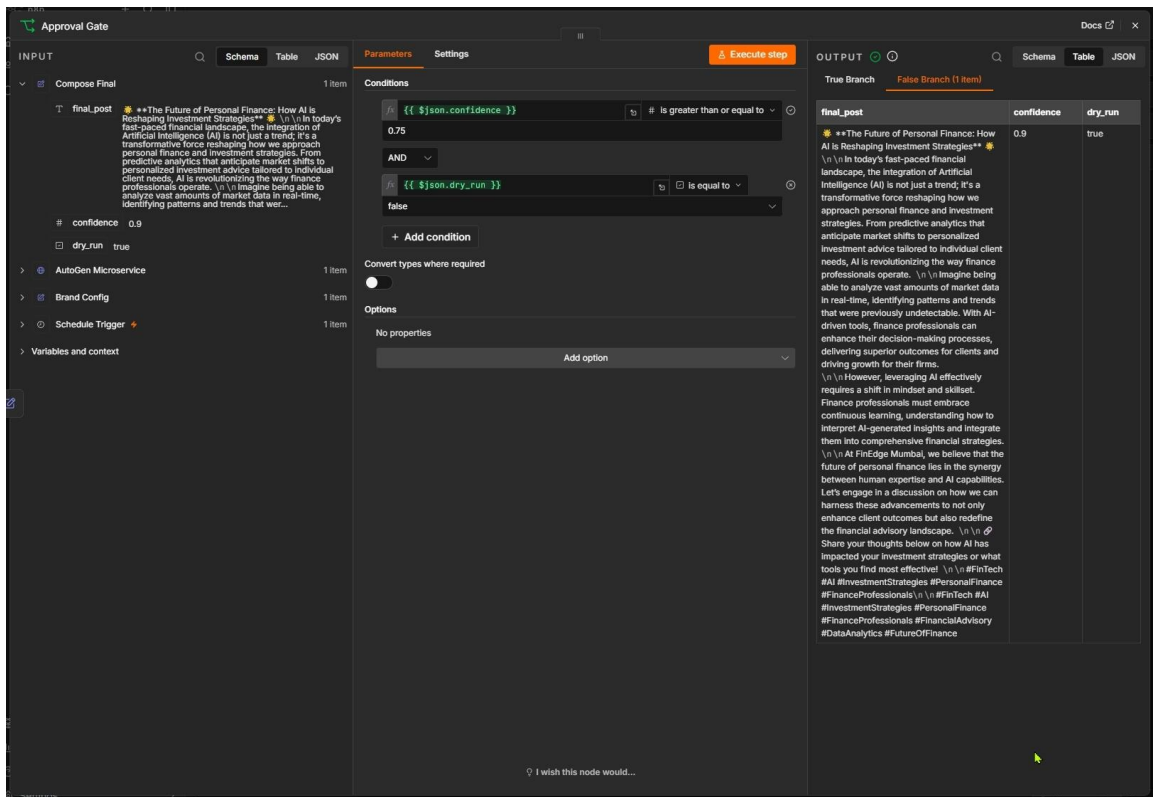


Figure 14 -- LinkedIn Mock Node

The LinkedIn node sits on the True branch. It is implemented as a mock HTTP Request node during development; in production it would use a LinkedIn OAuth credential to publish the post live.

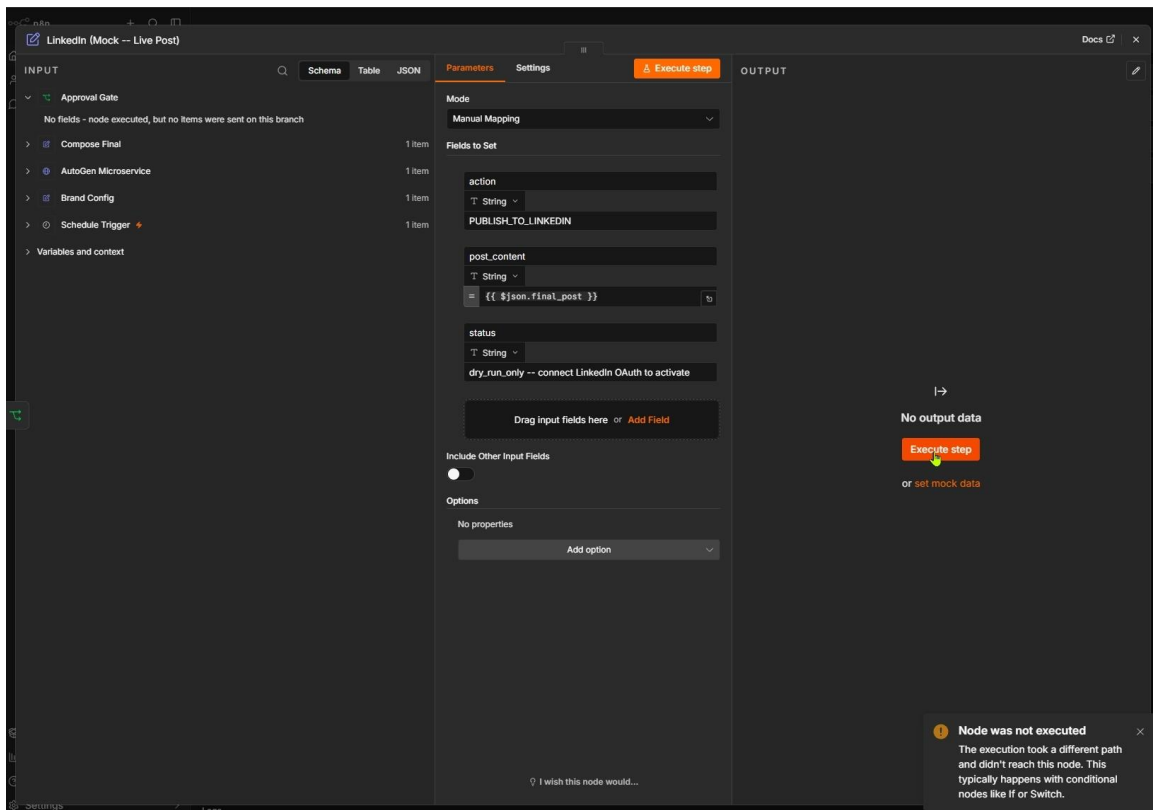


Figure 15 -- Slack Node

The Slack node sends the draft post via an Incoming Webhook. The message body includes the DRY RUN label, confidence score, and full post text so a human reviewer can approve or edit before live publishing.

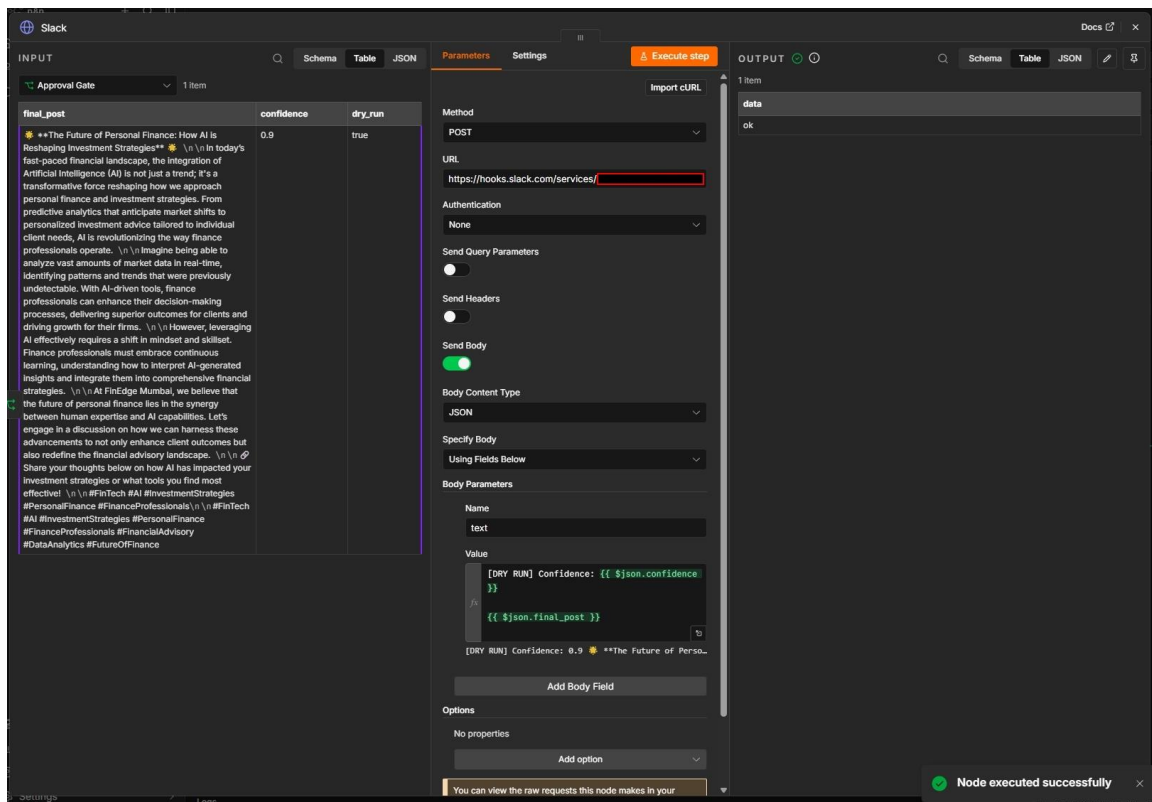


Figure 16 -- Logging Node

The Logging node POSTs run metadata (`final_post`, `confidence`, `dry_run`) to the FastAPI `/log` endpoint, which appends a row to `linkedin_log.csv`. Named-node expressions (`{{ 'Compose Final' }.item.json.* }`) are used to reach the original pipeline data regardless of which branch path arrived at this node.

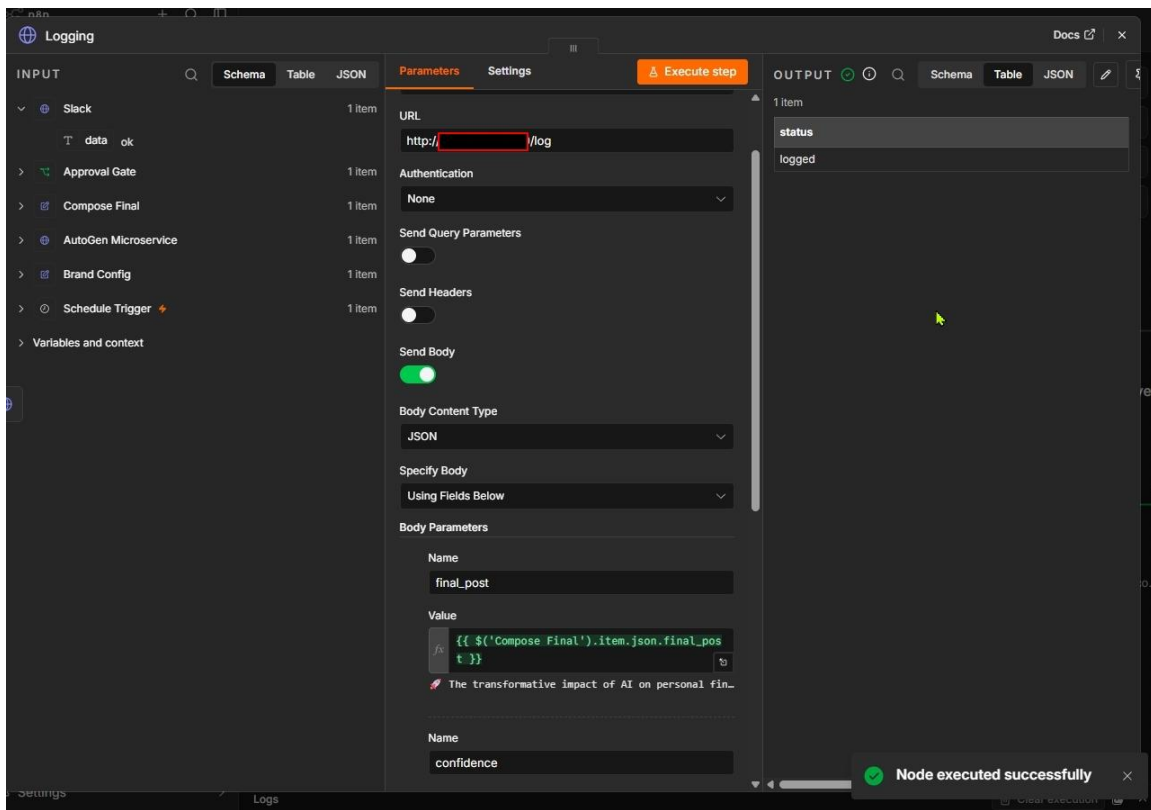


Figure 17 -- Node-Level Execute Test

Individual nodes were tested using the Execute Node button during construction. Green checkmarks confirm each node processed its input and produced the expected output before wiring to the next node.

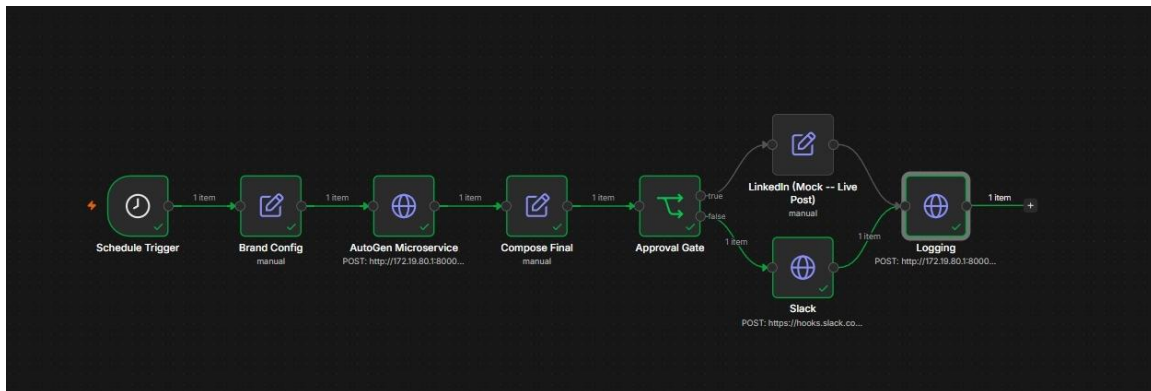


Figure 18 -- Full Workflow End-to-End Run

The complete workflow executed successfully from Schedule Trigger through Logging. All nodes on the active path (False branch via Slack) show green checkmarks, confirming the pipeline runs end-to-end without errors.


```
ROADMAP.md | linkedin_log.csv X
1 timestamp,draft,confidence,routed_to_dry_run
2 2026-06-28T23:33:53.326321+00:00,"🔗 The transformative impact of AI on personal finance is nothing short of revolutionary. As machine learning algorithms become increasingly sophisticated, they're reshaping investment strategies and risk assessment in ways we could only dream of a few years ago."
3
4 Gone are the days of one-size-fits-all financial advice. Today, AI enables hyper-personalization, allowing finance professionals to tailor investment recommendations based on individual risk profiles and market conditions. This level of customization not only enhances decision-making but also empowers consumers to take control of their financial futures.
5 2026-06-28T23:49:21.787587+00:00,"🔗 The Ethical Considerations of AI in Personal Finance: Balancing Innovation with Consumer Protection"
6
7 As the fintech landscape evolves, the integration of artificial intelligence (AI) into personal finance is not just a trend; it's a revolution. However, with great innovation comes great responsibility.
8
9 Finance professionals must navigate the complexities of AI's capabilities while ensuring consumer protection remains a priority. As we develop algorithms that personalize investment strategies",0.9,slack
10
```

Figure 19 -- Slack DRY RUN Message Received

The Slack channel received the AI-generated post with the DRY RUN label, confirming the approval gate correctly routes test traffic away from LinkedIn.

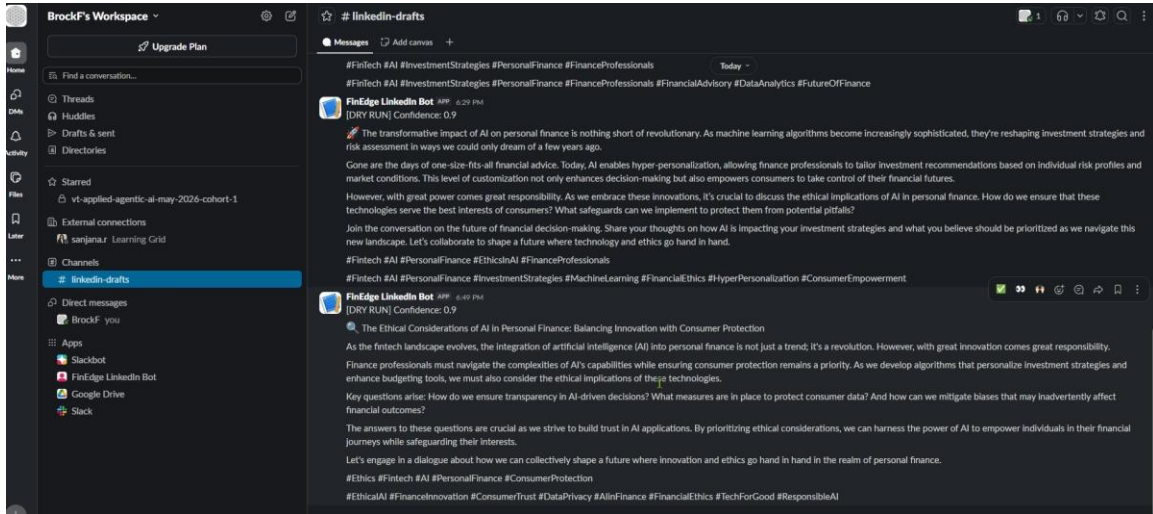


Figure 20 -- Slack Message Detail

Full view of the Slack message as it appears to a human reviewer -- draft text, confidence score, and routing metadata confirming the end-to-end Slack integration is working correctly.

```
ROADMAP.md | linkedin_log.csv X
1 timestamp,draft,confidence,routed_to_dry_run
2 2026-06-28T23:33:53.326321+00:00,"🔗 The transformative impact of AI on personal finance is nothing short of revolutionary. As machine learning algorithms become increasingly sophisticated, they're reshaping investment strategies and risk assessment in ways we could only dream of a few years ago."
3
4 Gone are the days of one-size-fits-all financial advice. Today, AI enables hyper-personalization, allowing finance professionals to tailor investment recommendations based on individual risk profiles and market conditions. This level of customization not only enhances decision-making but also empowers consumers to take control of their financial futures.
5 2026-06-28T23:49:21.787587+00:00,"🔗 The Ethical Considerations of AI in Personal Finance: Balancing Innovation with Consumer Protection"
6
7 As the fintech landscape evolves, the integration of artificial intelligence (AI) into personal finance is not just a trend; it's a revolution. However, with great innovation comes great responsibility.
8
9 Finance professionals must navigate the complexities of AI's capabilities while ensuring consumer protection remains a priority. As we develop algorithms that personalize investment strategies",0.9,slack
10
```